

Spring Testing Exercises

Exercise 1: Basic Unit Test for Service

CalculatorService.java

```
import org.springframework.stereotype.Service;
```

```
@Service
public class CalculatorService {

    public int add(int a, int b) {
        return a + b;
    }
}
```

CalculatorServiceTest.java

```
import static org.junit.jupiter.api.Assertions.assertEquals;
```

```
import org.junit.jupiter.api.Test;
```

```
public class CalculatorServiceTest {

    CalculatorService service = new CalculatorService();

    @Test
    void testAdd() {
        assertEquals(10, service.add(4, 6));
    }
}
```

Exercise 2: Mock Repository in Service Test

UserServiceTest.java

```
import static org.junit.jupiter.api.Assertions.*;
```

```
import static org.mockito.Mockito.*;
```

```
import java.util.Optional;
```

```
import org.junit.jupiter.api.Test;
```

```
import org.mockito.InjectMocks;
```

```

import org.mockito.Mock;
import org.mockito.MockitoAnnotations;

public class UserServiceTest {

    @Mock
    private UserRepository userRepository;

    @InjectMocks
    private UserService userService;

    public UserServiceTest() {
        MockitoAnnotations.openMocks(this);
    }

    @Test
    void testGetUserById() {

        User user = new User();
        user.setId(1L);
        user.setName("John");

        when(userRepository.findById(1L))
            .thenReturn(Optional.of(user));

        User result = userService.getUserById(1L);

        assertEquals("John", result.getName());
    }
}

```

Exercise 3: REST Controller Testing

UserControllerTest.java

```

import static org.mockito.Mockito.*;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;

```

```
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.test.web.servlet.MockMvc;
```

```
@WebMvcTest(UserController.class)
public class UserControllerTest {
```

```
    @Autowired
    MockMvc mockMvc;
```

```
    @MockBean
    UserService userService;
```

```
    @Test
    void testGetUser() throws Exception {
```

```
        User user = new User();
        user.setId(1L);
        user.setName("John");
```

```
        when(userService.getUserById(1L))
            .thenReturn(user);
```

```
        mockMvc.perform(get("/users/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.name")
                .value("John"));
```

```
    }
}
```

Exercise 4: Integration Test

```
import static org.junit.jupiter.api.Assertions.*;
```

```
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
```

```
@SpringBootTest
public class IntegrationTest {
```

```

@Autowired
private UserService userService;

@Test
void testIntegration() {

    User user = new User();
    user.setName("John");

    User saved = userService.saveUser(user);

    assertNotNull(saved);
}
}

```

Exercise 5: Test POST Endpoint

```

import static org.mockito.Mockito.*;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

import org.junit.jupiter.api.Test;

public class UserPostControllerTest {

    @Autowired
    MockMvc mockMvc;

    @MockBean
    UserService userService;

    @Test
    void testCreateUser() throws Exception {

        User user = new User();
        user.setId(1L);
        user.setName("John");

        when(userService.saveUser(any(User.class)))

```

```

        .thenReturn(user);

mockMvc.perform(post("/users")
    .contentType("application/json")
    .content("""
        {
            "name":"John"
        }
        """))
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.name")
        .value("John"));
    }
}

```

Exercise 6: Service Exception Handling

```

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

import java.util.Optional;
import java.util.NoSuchElementException;

import org.junit.jupiter.api.Test;

public class UserExceptionTest {

    @Test
    void testUserNotFound() {

        UserRepository repo = mock(UserRepository.class);

        when(repo.findById(1L))
            .thenReturn(Optional.empty());

        assertThrows(
            NoSuchElementException.class,
            () -> {
                throw new NoSuchElementException();
            });
    }
}

```

```
}  
}
```

Exercise 7: Custom Repository Query

```
import static org.junit.jupiter.api.Assertions.*;  
  
import java.util.List;  
  
import org.junit.jupiter.api.Test;  
import org.springframework.beans.factory.annotation.Autowired;  
  
@DataJpaTest  
public class RepositoryTest {  
  
    @Autowired  
    UserRepository repository;  
  
    @Test  
    void testFindByName() {  
  
        User user = new User();  
        user.setName("John");  
  
        repository.save(user);  
  
        List<User> users =  
            repository.findByName("John");  
  
        assertEquals(1, users.size());  
    }  
}
```

Exercise 8: Controller Exception Handler

```
import static org.mockito.Mockito.*;  
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;  
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;  
  
import java.util.NoSuchElementException;
```

```

import org.junit.jupiter.api.Test;

public class ExceptionHandlerTest {

    @Autowired
    MockMvc mockMvc;

    @MockBean
    UserService userService;

    @Test
    void testExceptionHandler() throws Exception {

        when(userService.getUserById(1L))
            .thenReturn(new NoSuchElementException());

        mockMvc.perform(get("/users/1"))
            .andExpect(status().isNotFound())
            .andExpect(content()
                .string("User not found"));
    }
}

```

Exercise 9: Parameterized Test

```

import static org.junit.jupiter.api.Assertions.*;

import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;

public class ParameterizedCalculatorTest {

    CalculatorService service =
        new CalculatorService();

    @ParameterizedTest
    @ValueSource(ints = {2,4,6,8,10})
    void testEvenNumbers(int number) {

```

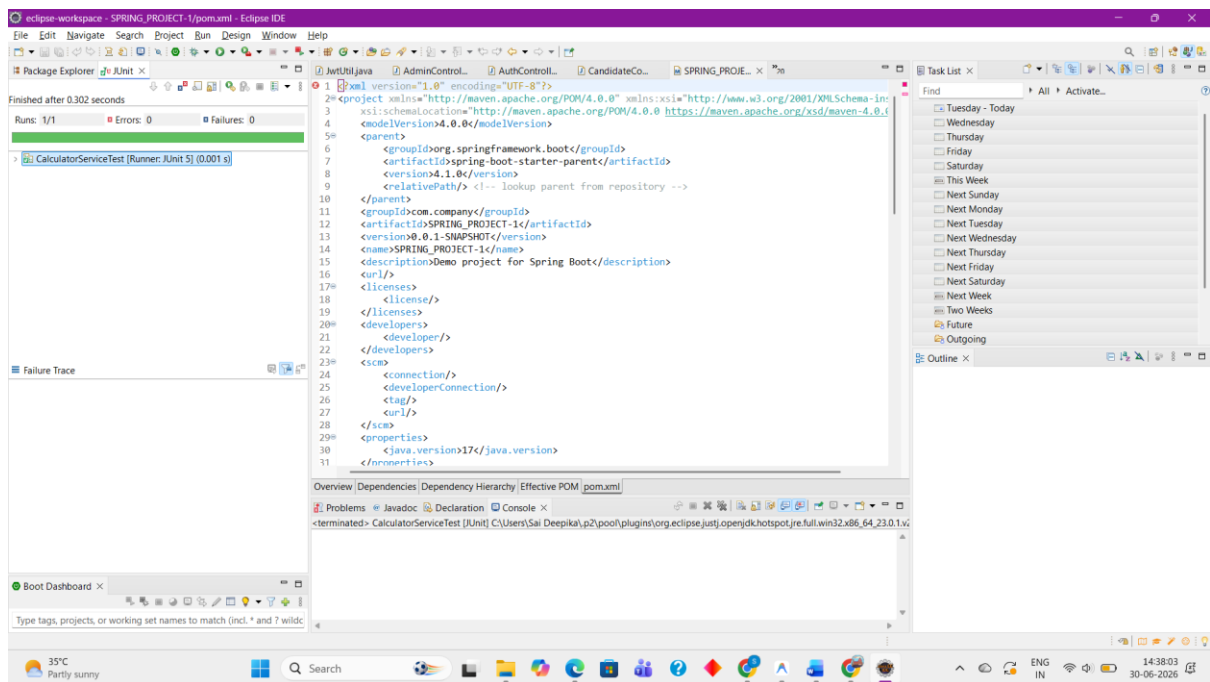
```
        assertEquals(0,
            number % 2);
    }
}
```

Required Maven Dependencies

Add these to your pom.xml:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
```

```
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>test</scope>
</dependency>
```

- spring_boot_project [BOOT] [devtools]
- SPRING_PROJECT-1 [boot] [devtools]
 - src/main/java
 - com.company.rms
 - CalculatorService.java
 - SpringProject1Application.java
 - User.java
 - UserController.java
 - UserRepository.java
 - UserService.java
 - src/main/resources
 - src/test/java
 - com.company.rms
 - CalculatorServiceTest.java
 - ExceptionHandlerTest.java
 - IntegrationTest.java
 - ParameterizedCalculatorTest.java
 - RepositoryTest.java
 - SpringProject1ApplicationTests.java
 - UserControllerTest.java
 - UserExceptionTest.java
 - UserPostControllerTest.java
 - UserServiceTest.java
 - src/test/resources
- JRE System Library [JavaSE-17]
- Maven Dependencies
- target/generated-sources/annotations
- JUnit 5
- Referenced Libraries
- target/generated-test-sources/test-annotations

